

Assignment - 06.

NAME

: Jothika.M

Reg No

: 192324235

Dept

: B.Tech (AI&DS)

Course Code

: CSA0389

Course Name

: Data Structures

Faculty Name

: Dr. Ashot Kumar

Assignment No : 6

Date of Submission : 21-08-2024

No. of pages : 06.

1. Write the algorithm for insertion sort and sort the following sequence:

(i) 3, 1, 4, 1, 5, 9, 2, 6, 5.

(ii) Explain the procedure for merge sort and perform the merge sort for the following inputs. Also, show the result for each step of iteration 64, 8, 216, 512, 27, 72, 9, 0, 1, 343, 125.

Algorithms:- for Insertion:-

1. Begin with the second element in the list.
2. Compare the current element to the previous elements.
3. Shift all larger element one position to the right.
4. Insert the current element into its correct position.
5. Repeat steps 2-4 for each element in the list until the entire list is sorted.

Sorting the Sequence:-

Sequence: 3, 1, 4, 1, 5, 9, 2, 6, 5

3	1	4	1	5	9	2	6	5
---	---	---	---	---	---	---	---	---

Compare 3 & 1, 3 > 1
Swap 3, 1

1	3	4	1	5	9	2	6	5
---	---	---	---	---	---	---	---	---

Compare 4 & 1, 4 > 1
Swap 4, 1

1	3	1	4	5	9	2	6	5
---	---	---	---	---	---	---	---	---

Compare 3 & 1, 3 > 1
Swap 3, 1

1	1	3	4	5	9	2	6	5
---	---	---	---	---	---	---	---	---

Compare 9 & 2, 9 > 2 Swap 9, 2

1 1 3 4 5 2 9 6 5

Compare 5 & 2, $5 > 2$ Swap 5, 2

1 1 3 4 2 5 9 6 5

Compare 4 & 2, $4 > 2$ Swap 4, 2

1 1 3 2 4 5 9 6 5

Compare 3 & 2, $3 > 2$ Swap 3, 2

1 1 2 3 4 5 9 6 5

Compare 9 & 6, $9 > 6$ Swap 9, 6

1 1 2 3 4 5 6 9 5

Compare 9 & 5, $9 > 5$ Swap 9, 5

1 1 2 3 4 5 6 5 9

Compare 6 & 5, $6 > 5$ Swap 6, 5

1 1 2 3 4 5 5 6 9

Sorted.

Sorted Sequence: 1, 1, 2, 3, 4, 5, 5, 6, 9.

Merge Sort Procedure.

* Split the list into halves until each sublist has one element.

* Combine the sublists to produce new sorted sublists until there is one sorted list.

Merge Sort with 64, 8, 216, 512, 27, 729, 0, 1, 343, 125.

1) Initial Split:

* [64, 8, 216, 512, 27] and [729, 0, 1, 343, 125]

2. Further Split:

- $[64, 8]$ and $[216, 512, 29]$
- $[729, 0]$ and $[1, 343, 125]$

3. Further Split:

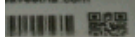
- $[64]$ and $[8]$
- $[216]$ and $[512, 29]$
- $[729]$ and $[0]$
- $[1]$ and $[343, 125]$

4. Merge.

- Merge $[64]$ and $[8] \rightarrow [8, 64]$
- Merge $[512, 29] \rightarrow [29, 512]$
- Merge $[216]$ and $[29, 512] \rightarrow [29, 216, 512]$
- Merge $[0]$ and $[729] \rightarrow [0, 729]$
- Merge $[125, 343] \rightarrow [125, 343]$
- Merge $[1]$ and $[125, 343] \rightarrow [1, 125, 343]$

5. Final Merge:

- Merge $[8, 64]$ and $[29, 216, 512]$
 $\Rightarrow [8, 29, 64, 216, 512]$
- Merge $[0, 729]$ and $[1, 125, 343]$
 $\Rightarrow [0, 1, 125, 343, 729]$
- Merge $[8, 29, 64, 216, 512]$ and $[0, 1, 125, 343, 729]$



$\Rightarrow [0, 1, 8, 27, 64, 125, 216, 343, 512, 729]$

Sorted list: $0, 1, 8, 27, 64, 125, 216, 343, 512, 729$.

2). Draw the concept map of partitioning in quick sort, try to write an algorithm for it, which is as follows, & develop a program considering the steps.

Step 1 - Choose the highest index value as pivot.

Step 2 - Take two variables to point left and right of the list excluding pivot.

Step 3 - Left points to the low index.

Using elements your own.

Algorithm:-

* Select the element at the highest index as the pivot.

* Set 'left' to the low index and 'right' to the high index - 1.

* Move 'left' rightwards and 'right' leftwards until 'left' is greater than or equal to 'right' swapping elements as needed.

* Swap the pivot with the element at the left pointer position.

Return the index of the pivot element.

Program:-
=====

```
#include <stdio.h>
```

```
int main() {
```

```
    int arr[] = {64, 18, 216, 512, 27, 729, 0, 1, 343, 125};
```

```
    int n = sizeof(arr) / sizeof(arr[0]);
```

```
    int low = 0, high = n - 1;
```

```
    while (low < high) {
```

```
        int pivot = arr[high];
```

```
        int left = low;
```

```
        int right = high - 1;
```

```
        while (left <= right) {
```

```
            while (left <= right && arr[left] < pivot) {
```

```
                left ++;
```

```
            }
```

```
            while (right >= low && arr[right] > pivot) {
```

```
                right --;
```

```
            }
```

```
            int (left < right) {
```

```
                int temp = arr[left];
```

```
                arr[left] = arr[right];
```

```
                arr[right] = temp;
```

```
                left ++;
```

```

        right--;
    }
}
int temp = arr[left];
arr[left] = arr[right];
arr[right] = temp;
    high = left - 1;
    if (high < low) {
        low = left + 1;
        high = n - 1;
    }
}
printf("Sorted array:");
for (int i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
printf("\n");
return 0;
}

```

OUTPUT:

Sorted array: 0, 1, 8, 27, 64, 125, 216, 343, 512, 729.